CHAPTER 12

AI Agents: Reasoning, Tools, Planning, and Context

chain-of-thought • ReAct • agentic workflows • function calling • MCP • evaluation

> **Central question**
>
> How do we move from a language model that produces responses to a system that can plan, use tools, react to observations, and reliably complete tasks? The answer is a controlled loop that combines reasoning, action, environmental feedback, evaluation, and human authority.

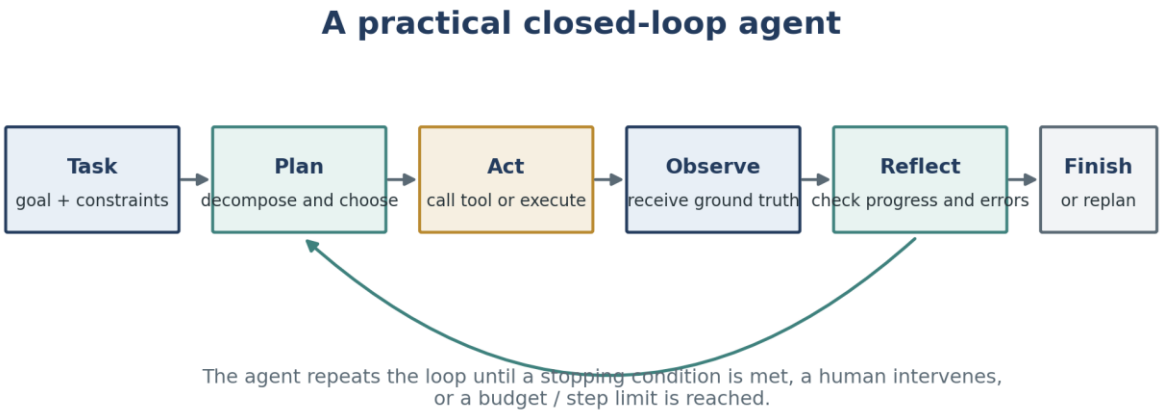

Figure 1. A synthesized agent loop based on the assigned sources. The defining change is not merely longer reasoning; it is repeated interaction with an environment that can confirm, contradict, or redirect the model.

Learning objectives

- Distinguish a fixed LLM workflow from a model-directed agent.
- Explain how chain-of-thought prompting elicits intermediate reasoning without adding external grounding.
- Describe the ReAct thought-action-observation loop and its advantages over reasoning-only or acting-only approaches.
- Select among prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, and autonomous agents.
- Describe tool inventories, function calling, plan validation, reflection, and human approval checkpoints.
- Explain how MCP standardizes connections among AI hosts, clients, servers, resources, prompts, and tools.
- Identify planning, tool, grounding, efficiency, security, and false-completion failure modes.
- Design an evaluation and control strategy for a tool-using agent.

> **Source-date note**
>
> The research papers report 2022-era experiments, the engineering guidance was published in 2024-2025, and the MCP material uses the official 2026-07-28 protocol documentation. Use the papers for durable mechanisms and evidence, not current model rankings or parameter thresholds.

1 What makes a system an agent?

An agent is best understood as a system that receives information about an environment, selects actions, observes the consequences, and continues until a goal or stopping condition is reached. The model may supply planning and judgment, but the complete agent also includes tools, state, permissions, control logic, and evaluation. (Huyen, 2025; Anthropic, 2024)

1.1 Core definition

- Environment: the external setting in which the system operates—such as a file system, website, database, simulated household, code repository, or business application.
- Observation: information returned from the environment, including tool results, errors, test output, search results, or user feedback.
- Action: an operation the agent can perform, such as retrieve, calculate, edit, navigate, send, or stop.
- Goal and constraints: what success means and what boundaries must be respected.
- Planner or policy: the mechanism that chooses what to do next from the current context.
- State or memory: the record of prior observations, actions, intermediate results, unresolved subgoals, and relevant context.
- Stopping condition: a completion test, explicit handoff, maximum step count, budget, timeout, or safety halt.

> **Compact definition**
>
> Agentic system = model + environment + tools + state + control loop + evaluation + authority boundaries.

1.2 Workflow versus agent

Dimension	Workflow	Agent
Control path	Predefined in code	Dynamically selected by the model
Best fit	Stable, well-specified tasks	Open-ended tasks with unpredictable steps
Predictability	Higher	Lower
Flexibility	Limited to designed branches	Can adapt plans and tool use
Cost / latency	Usually bounded	May grow with each loop iteration
Primary risk	Brittle handling of unanticipated cases	Compounding errors, loops, and unsafe actions

Anthropic uses “agentic systems” as an umbrella term, then separates workflows from agents. A workflow can contain several LLM calls and tools without giving the model control over the overall path. An agent is model-directed: it decides what steps and tools are needed from the evolving context. (Anthropic, 2024)

1.3 When agent autonomy is warranted

- Use a simple model call when the task can be solved with a clear prompt, retrieval, examples, or one tool invocation.
- Use a workflow when the subtasks, gates, and order can be specified in advance.
- Use an agent when the number and type of required steps cannot be predicted, environmental feedback changes the plan, and success can be checked during execution.
- Avoid an autonomous loop when failures are irreversible, success cannot be verified, permissions cannot be constrained, or the expected value does not justify the latency and cost.

1.4 The augmented LLM as the building block

The common foundation is an LLM augmented with retrieval, tools, and memory. The model generates queries, selects tools, decides what context matters, and interprets the results. The system around the model must make those capabilities easy to understand, reliably callable, and observable. (Anthropic, 2024)

Agent component	Purpose	Example failure if weak
Prompt / policy	Defines goals, rules, priorities, and completion criteria	The agent optimizes the wrong objective.
Tool interface	Exposes available actions and their parameters	The model calls the wrong tool or invents arguments.
Observation channel	Returns environmental ground truth	The agent continues from stale or ambiguous state.
Memory / state	Tracks progress and important facts	The agent repeats work or loses a completed subgoal.
Evaluator	Checks quality, safety, and completion	The agent falsely declares success.
Permission layer	Limits consequences	A planning error becomes an irreversible action.

2 Reasoning before action: chain-of-thought prompting

Chain-of-thought (CoT) prompting supplies a few demonstrations whose outputs include intermediate natural-language reasoning steps before the final answer. The technique does not retrain the model; it changes the inference-time examples and output pattern. (Wei et al., 2022)

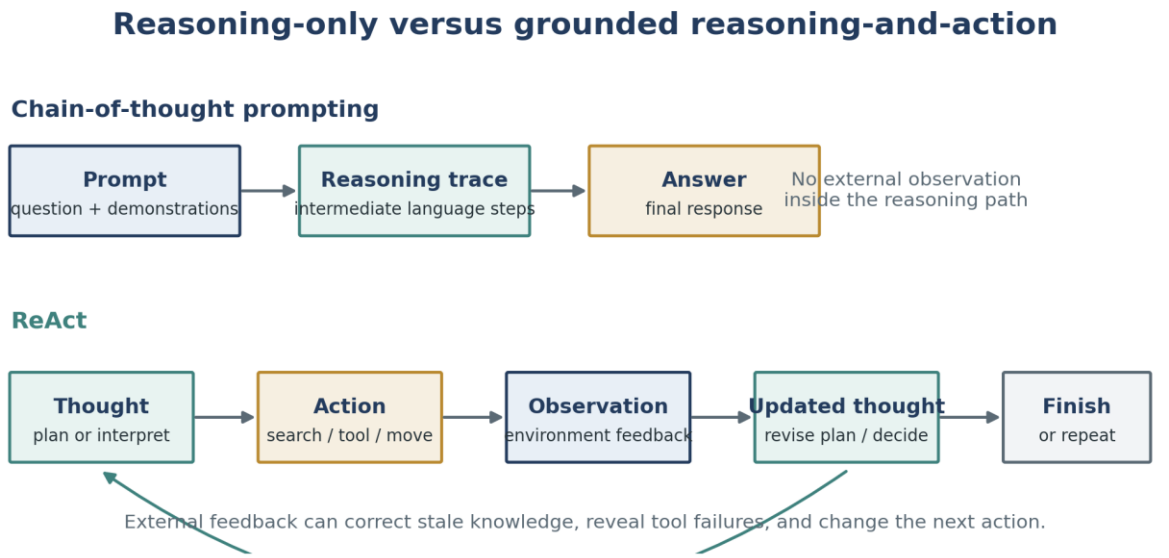

Figure 2. Chain-of-thought creates an internal reasoning trajectory. ReAct adds actions and observations, converting static reasoning into a feedback-driven process.

2.1 Why intermediate steps can help

- Decomposition: a multi-step problem is divided into manageable intermediate steps.
- Variable computation: harder problems can consume more generated tokens than easy problems.
- Debuggability: a visible rationale can reveal where a reasoning path became incorrect, although it is not a complete account of the network's internal computation.
- Task generality: the same natural-language mechanism can be applied to arithmetic, commonsense, symbolic manipulation, and planning-oriented tasks.
- Few-shot elicitation: sufficiently capable off-the-shelf models can display the behavior from demonstrations alone.

2.2 Evidence from the assigned paper

Finding	Reported evidence	Interpretation
Large GSM8K gain	PaLM 540B rose from about 18% with standard prompting to about 57% with eight CoT exemplars, exceeding the cited prior best of 55%.	Intermediate reasoning can unlock performance that direct-answer prompting does not reveal.
Scale dependence	Smaller models often produced fluent but illogical traces; strong gains appeared around the largest model scales studied.	CoT was an emergent prompting capability in these experiments, not a universal threshold.
Complexity dependence	Gains were largest for multi-step problems and small or negative on easy single-step subsets.	Reasoning traces are most useful when decomposition is actually needed.
Commonsense results	PaLM 540B reached 75.6% on StrategyQA versus a cited 69.4% prior best, and 95.4% on sports understanding versus an 84% unaided-human reference.	The method extends beyond arithmetic, though gains vary by task.

Finding	Reported evidence	Interpretation
Symbolic generalization	CoT improved performance on longer unseen sequences in letter-concatenation and coin-flip tasks.	Explicit steps can help length generalization when the procedure is demonstrated.

2.3 Ablations: what did not explain the gain

- Equation only: producing only a mathematical expression did not explain the GSM8K improvement; semantic interpretation still required natural-language steps.
- Variable compute only: generating placeholder tokens to spend more computation performed near the baseline, so extra tokens alone were insufficient.
- Reasoning after the answer: placing an explanation after the answer performed near the baseline, indicating that the sequential path before the answer mattered.
- Prompt style: different annotators, concise rationales, exemplar sets, and exemplar orders changed performance, but the CoT prompts generally remained better than standard prompting.

2.4 Limitations of CoT as an agent mechanism

- A plausible rationale is not proof that the internal model computation followed the same path.
- The reasoning trace can contain semantic errors, missing steps, unsupported facts, or a correct answer reached by coincidence.
- Reasoning-only prompting cannot inspect the world during the trace; stale or fabricated facts may propagate across later steps.
- Long traces increase token cost and latency, and can consume context that would otherwise hold evidence or instructions.
- The original results depended strongly on model capability; the reported parameter sizes should not be treated as current universal cutoffs.

> **Key transition**
>
> CoT shows that language can organize multi-step reasoning. Agents require the next step: let the reasoning trigger actions, then let real observations revise the reasoning.

3 ReAct: reasoning and acting in a loop

ReAct interleaves reasoning traces with task-specific actions and environmental observations. Reasoning helps create and update a plan; actions obtain external information or change the environment; observations provide feedback for the next thought. The authors summarize this as “reason to act” and “act to reason.” (Yao et al., 2023)

3.1 The thought-action-observation trajectory

1. Thought: identify the next information need, subgoal, exception, or plan revision.
2. Action: call an allowed operation such as search, lookup, navigation, manipulation, or finish.
3. Observation: receive the result from the external environment.
4. Update: integrate the observation, track progress, and choose the next thought or action.
5. Terminate: finish only when the answer or environmental goal satisfies the completion condition.

For knowledge-intensive question answering, ReAct uses dense thought-action-observation cycles. For long-horizon decision tasks, the paper uses sparse reasoning at the moments where goal decomposition, progress tracking, commonsense, or exception handling is most useful.

3.2 Standard, CoT, Act-only, and ReAct

Method	Reasoning trace	External action	Observation feedback	Typical weakness
Standard	No	No	No	Direct answers can be unsupported or shallow.
CoT	Yes	No	No	Structured reasoning can propagate hallucinated facts.
Act-only	No	Yes	Yes	The system may call tools without a coherent plan or state model.
ReAct	Yes	Yes	Yes	More grounded, but vulnerable to bad retrieval, loops, and reasoning rigidity.

3.3 Reported benchmark results

Task	Selected result	What it shows
HotpotQA	ReAct: 27.4 EM; CoT: 29.4; ReAct→CoT-SC: 35.1.	Grounding alone did not beat reasoning-only, but hybrid fallback performed best among the prompting methods reported.
FEVER	ReAct: 60.9% accuracy; CoT: 56.3; CoT-SC→ReAct: 64.6.	External retrieval was especially useful when small factual differences determined the label.
ALFWorld	Best ReAct: 71% success; Act: 45%; BUTLER: 37%.	Sparse reasoning improved subgoal decomposition, progress tracking, and exploration.
WebShop	ReAct: 40.0% success; Act: 30.1%; IL: 29.1%; IL+RL: 28.7%; expert human: 59.6%.	One-shot reasoning plus action improved product selection, but remained well below expert behavior.

3.4 Groundedness versus flexibility

The paper's human analysis found different failure profiles. In the sampled HotpotQA failures, CoT was dominated by hallucinated reasoning or facts, while ReAct had no hallucination cases in that failure category but had more reasoning errors and search-result failures. ReAct can become repetitive, fail to escape a loop, or derail when retrieval returns uninformative evidence. (Yao et al., 2023)

Observed failure category	ReAct	CoT	Design lesson
False-positive success with hallucinated trace / facts	6%	14%	External evidence reduced unsupported success cases.

Observed failure category	ReAct	CoT	Design lesson
Reasoning error among failures	47%	16%	The structured action loop can constrain flexible reasoning and create repetition.
Search-result error among failures	23%	—	Retrieval quality and query reformulation are critical.
Hallucination among failures	0%	56%	Reasoning-only methods need grounding or verification.

> **Hybrid lesson**
>
> Internal reasoning and external knowledge are complementary. The best reported prompting results used fallback combinations between self-consistent CoT and ReAct rather than treating either method as sufficient alone.

4 Production patterns for agentic systems

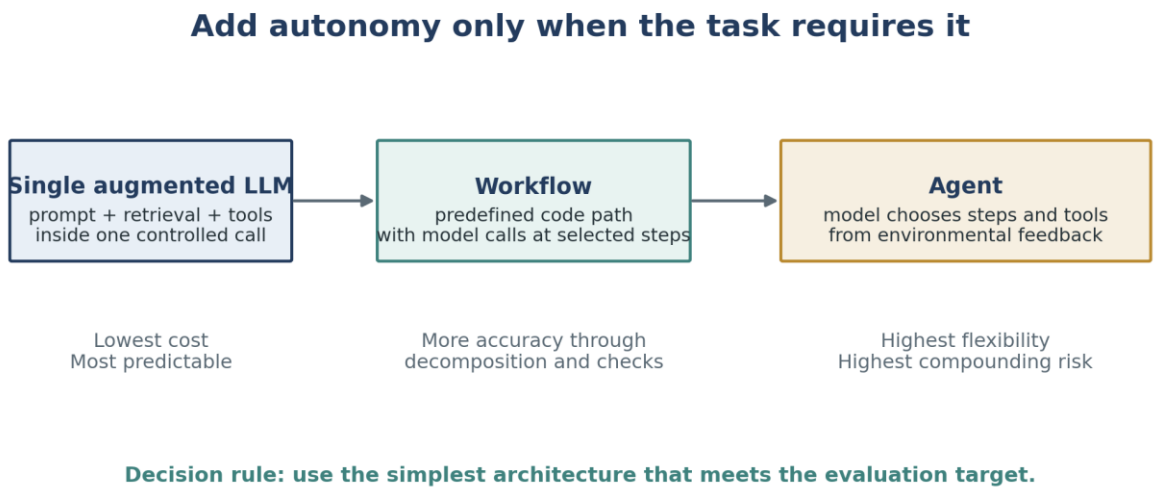

Figure 3. Anthropic's production guidance recommends increasing architectural complexity only when evaluations show that simpler approaches are insufficient.

4.1 Common workflow patterns

Pattern	Control structure	Best use	Primary caution
Prompt chaining	Output of one call becomes input to the next; optional gates between steps.	Tasks that decompose cleanly into a fixed sequence.	Latency grows with each stage; a bad early output contaminates later stages.
Routing	Classify the input, then send it to a specialized prompt, model, or toolset.	Distinct categories such as billing, technical support, or easy versus hard requests.	Misclassification sends the task down the wrong path.

Pattern	Control structure	Best use	Primary caution
Parallelization	Run independent subtasks or multiple attempts concurrently, then aggregate.	Speed through sectioning or confidence through voting.	Higher cost; aggregation rules can hide disagreement.
Orchestrator-workers	A central model dynamically creates and delegates subtasks, then synthesizes results.	Complex work whose subtasks cannot be predicted in advance.	Delegation quality and result synthesis become new failure points.
Evaluator-optimizer	A generator produces a result; an evaluator critiques it; the loop repeats.	Tasks with clear quality criteria and measurable gains from revision.	Can loop, over-optimize style, or incur large token cost.
Autonomous agent	The model chooses tools and next steps from observations until stopping.	Open-ended tasks with verifiable progress and trusted environments.	Compounding errors, unbounded cost, and permission risk.

4.2 Parallelization versus orchestrator-workers

Parallelization divides work using predefined independent branches. Orchestrator-workers uses a central LLM to decide what branches are needed for the specific input. The diagrams can look similar, but the latter transfers subtask design to the model and therefore provides more flexibility at the cost of more uncertainty. (Anthropic, 2024)

4.3 Frameworks and abstraction

- Frameworks reduce boilerplate for model calls, tool schemas, parsing, tracing, and chaining.
- Extra abstraction can hide the actual prompts, responses, and failure points, making debugging harder.
- Start with direct APIs and simple components when possible; use a framework only when its orchestration and observability benefits exceed its complexity.
- Understand the generated code paths and tool behavior instead of relying on assumptions about the framework.

4.4 Agent-computer interface (ACI)

Tool definitions are prompts to the model. Names, descriptions, parameter schemas, examples, edge cases, and error messages directly affect planning accuracy. Anthropic recommends investing in agent-computer interfaces with the same seriousness applied to human-computer interfaces. (Anthropic, 2024)

- Prefer formats that are natural for the model and avoid unnecessary escaping or bookkeeping.
- Use clear, distinct parameter names and document boundaries between similar tools.
- Include valid examples, invalid examples, edge cases, and completion conditions.
- Make mistakes difficult: constrain schemas, require absolute paths where appropriate, validate arguments, and design idempotent operations.
- Test the tool interface with many representative and adversarial requests, then revise the interface—not only the system prompt.

5 Tools, function calling, planning, and reflection

5.1 Three broad tool categories

Category	Purpose	Examples	Risk focus
Knowledge augmentation	Retrieve context the model does not reliably hold.	Search, files, databases, email readers, private knowledge bases.	Source quality, access control, staleness, and prompt injection.
Capability extension	Delegate tasks that specialized software performs better.	Calculator, code interpreter, unit converter, renderer, transcription.	Code injection, incorrect execution, and over-trusting tool output.
Write actions	Change the environment or trigger external consequences.	Send email, update records, merge code, issue refund, schedule event.	Authorization, reversibility, confirmation, and auditability.

Read actions primarily help the agent perceive the environment. Write actions alter the environment and therefore require stronger controls. Tool access should follow least privilege: expose only the operations and data needed for the task. (Huyen, 2025)

5.2 Function calling is a proposal-execution protocol

1. Declare the tool inventory, including the tool name, purpose, parameters, types, and constraints.
2. Give the model the subset of tools relevant to the current request.
3. The model proposes a structured tool call and argument values.
4. Application code validates permissions, schema, parameter values, and policy conditions.
5. The application—not the language model—executes the tool.
6. The tool result is returned as an observation and becomes context for the next model decision.
7. The loop continues or terminates according to a completion test.

> **Critical distinction**
>
> A model generating a syntactically valid tool call does not guarantee that the selected tool, parameter values, timing, or intended consequence is correct.

5.3 Plan generation, validation, execution, and reflection

A task combines a goal with constraints. Planning identifies a sequence of actions that could reach the goal efficiently. Huyen recommends separating planning from execution so invalid or wasteful plans can be rejected before they consume time, money, or permissions. (Huyen, 2025)

1. Generate: propose one or more candidate plans or next-step plans.
2. Validate: reject unavailable tools, impossible actions, excessive step counts, missing approvals, or plans that violate constraints.
3. Execute: perform an approved action and capture the full output or error.
4. Reflect: evaluate whether the action moved the system toward the goal and whether the plan must change.
5. Complete or replan: finish only when explicit success criteria are satisfied.

- Validation may use deterministic heuristics, a separate evaluator model, domain-specific checks, or human review.
- Human approval should be required before risky operations such as database updates, financial actions, public communication, or code merges.
- A plan does not need to cover the entire task. Short-horizon planning allows observations to correct the next subplan.

5.4 Planning granularity and control flow

Design choice	Advantage	Disadvantage
Detailed executable plan	Easier to execute and validate step by step.	Harder to generate; tightly coupled to the current tool API.
High-level natural-language plan	More reusable across tools and easier for models to generate.	Requires a translator or lower-level planner before execution.
Hierarchical planning	Combines stable high-level goals with adaptable local plans.	Adds components and evaluation boundaries.
Sequential flow	Simple dependency handling.	Slow when independent work could run concurrently.
Parallel flow	Reduces elapsed time and supports multiple perspectives.	Consumes more resources and needs result aggregation.
Conditional / routing flow	Adapts to prior outputs.	Depends on accurate classification or condition evaluation.
Loop	Supports iteration and recovery.	Requires reliable stopping conditions and loop detection.

5.5 Compounding reliability

> **A simple reliability illustration**
>
> Under an independence approximation, a process that succeeds 95% of the time at each step succeeds across 10 steps only about 59.9% of the time (0.95^{10}), and across 100 steps about 0.6% of the time (0.95^{100}). Longer horizons therefore require checkpoints, retries with limits, deterministic validation, and recovery—not merely a strong base model.

- Reduce unnecessary steps and combine operations that are always used together.
- Prefer tools that return explicit, structured errors over ambiguous natural-language failures.
- Persist checkpoints so the agent can resume rather than restart an entire long task.
- Use budgets for tokens, time, tool calls, retries, and monetary cost.

5.6 Multi-agent systems

A planner, validator, executor, and evaluator can each be implemented as separate components. This division can improve specialization and auditability, but it also creates communication overhead and new failure interfaces. “Multi-agent” should describe a useful decomposition, not complexity added for its own sake.

6 Model Context Protocol (MCP)

MCP is an open standard for connecting AI applications to external data sources, tools, and workflows. The official documentation compares it to a USB-C port: the protocol standardizes the connection, while the devices and applications retain their own capabilities and policies. (Model Context Protocol, 2026)

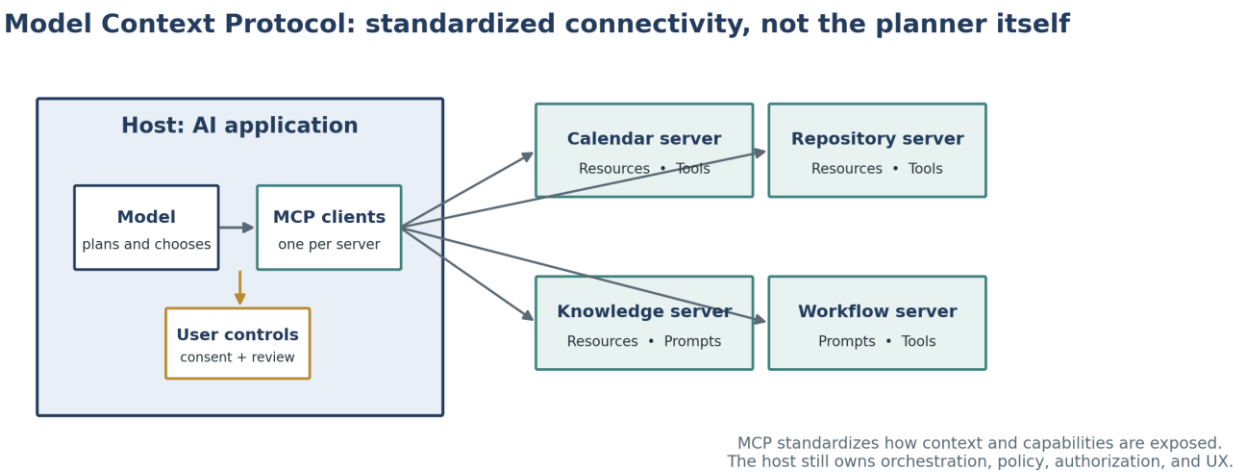

Figure 4. MCP separates the AI host from connected servers. It standardizes discovery and invocation of context and capabilities; it does not replace the agent's planner, authorization layer, or user interface.

6.1 Core roles

Role	Responsibility	Example
Host	The AI application that manages the user experience, model, policies, and connections.	An assistant, IDE, desktop app, or enterprise chat interface.
Client	A connector within the host that communicates with one MCP server.	The host-side client for a calendar or repository server.
Server	A service that exposes context or capabilities through MCP.	A database, filesystem, design tool, calendar, or domain workflow.

6.2 Capabilities in the 2026-07-28 specification

- Resources: contextual data that a user or model can read and use.
- Prompts: templated messages and workflows that can be surfaced to users.
- Tools: functions the model can request to execute.
- Elicitation: a client-side capability that lets a server request additional user information through the host.
- Base protocol: JSON-RPC 2.0 messages with stateless, self-contained requests and per-request capability information in the 2026-07-28 version.
- Optional extensions: Tasks for long-running work, Skills over MCP for structured agent instructions, and MCP Apps for interactive UI elements.

6.3 What MCP does—and does not—solve

MCP helps standardize	MCP does not automatically provide
How tools, resources, and prompts are described and accessed.	Correct planning or task decomposition.
Interoperability across hosts and servers.	Safe authorization or appropriate user consent.
Discovery of server capabilities.	Reliable tool selection and parameter values.
A common communication protocol.	Evaluation, stopping conditions, or error recovery.
Composable integrations.	Trust in server data, descriptions, or code.

6.4 Example interaction

1. A user asks an AI host to complete a task that requires external data or action.
2. The host exposes approved MCP server capabilities to the model through its clients.
3. The model selects a resource or proposes a tool call.
4. The host checks policy, authorization, user consent, and arguments.
5. The client sends the request to the server and returns the result as an observation.
6. The model interprets the result, continues the plan, requests approval for a consequential step, or finishes.

6.5 MCP security principles

- User consent and control: users should understand what data is accessed and what actions are authorized.
- Data privacy: do not expose or retransmit user data without explicit permission and appropriate access controls.
- Tool safety: treat tool invocation as arbitrary code execution risk and require explicit authorization for consequential operations.
- Trust boundaries: tool descriptions, annotations, and returned content can be untrusted unless the server itself is trusted.
- Implementation responsibility: MCP standardizes the protocol; hosts and servers must still implement robust authentication, authorization, logging, and data protection.

7 Failure modes, evaluation, and safety

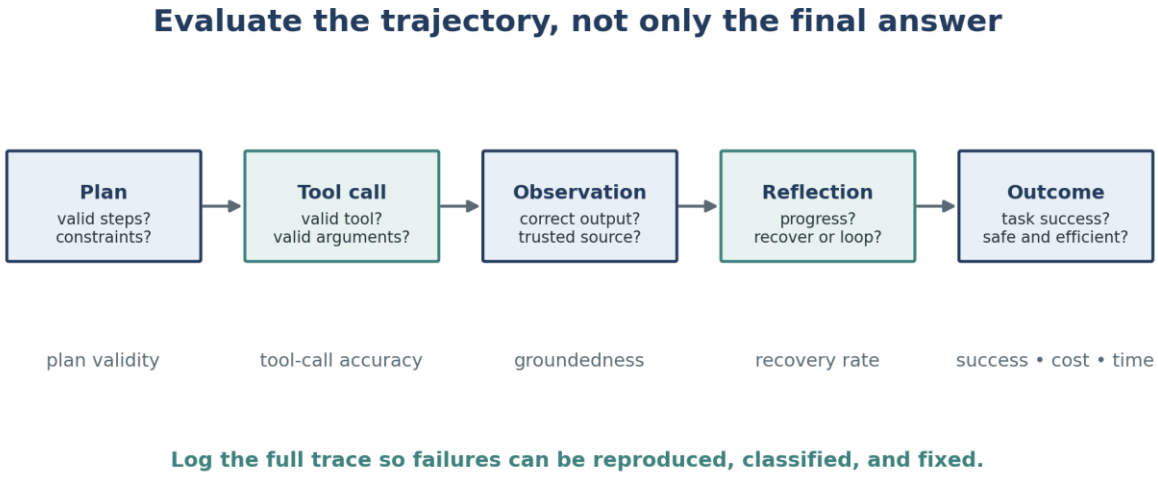

Figure 5. A final answer can look correct while the trajectory contains invalid tools, unsupported observations, unrecovered errors, excessive cost, or unsafe actions. Evaluation must cover every stage.

7.1 Planning failures

- Invalid tool: the plan requests an operation that is not available.
- Invalid schema: the tool exists, but the arguments have the wrong names, types, or number.
- Incorrect values: the call is syntactically valid but uses the wrong entity, date, amount, path, or scope.
- Goal failure: the plan solves the wrong problem or ignores a required constraint.
- Deadline or budget failure: the plan is technically valid but too slow or expensive to be useful.
- False completion: reflection concludes that the goal has been reached when required work is missing.

7.2 Tool and observation failures

- The correct tool returns incorrect, incomplete, stale, biased, or malicious content.
- A natural-language planner is translated into the wrong executable command.
- The tool reports failure ambiguously, causing the model to treat the operation as successful.
- The tool inventory is missing a capability that a competent human would use.
- The agent ignores a useful observation or gives unsupported parametric knowledge priority over the result.
- Untrusted content injects instructions that attempt to redirect the agent or exfiltrate information.

7.3 Agent-specific operational failures

- Looping: the agent repeats thoughts and actions without progress.
- Context exhaustion: long traces, examples, tool outputs, and documents crowd out essential instructions or state.
- State drift: the agent loses track of completed work, current objects, or the latest external condition.
- Over-delegation: the agent calls a powerful tool when a deterministic local operation would be safer.
- Permission escalation: a broad tool or credential makes a minor planning mistake consequential.
- Irreversible action: the system lacks dry-run, confirmation, rollback, or idempotency controls.

7.4 Evaluation dimensions and metrics

Dimension	Example metrics	Question answered
Task success	Completion rate, exact match, test pass rate, constraint satisfaction.	Did the system accomplish the requested goal?
Plan quality	Valid-plan rate, attempts to first valid plan, unnecessary steps.	Was the proposed route feasible and efficient?
Tool use	Valid tool rate, schema accuracy, parameter-value accuracy.	Did the agent invoke the correct capability correctly?
Grounding	Evidence support, source freshness, citation correctness.	Do observations support the claims and decisions?
Recovery	Loop rate, retry success, recovery after tool errors.	Can the system recognize and correct failure?
Efficiency	Steps, tokens, wall-clock time, tool latency, monetary cost.	Is the result worth the resources consumed?
Safety	Unauthorized-action rate, policy violations, injection success, data exposure.	Did the system stay within authority and trust boundaries?
Human factors	Approval burden, override rate, comprehensibility of trace.	Can users understand and control the system?

7.5 Layered controls

- Sandbox code execution and isolate network, filesystem, and credential access.
- Use tool allowlists, per-request tool subsets, strict schemas, and server-side validation.
- Separate read and write capabilities; require confirmation or policy approval for write actions.
- Use dry-run modes, transaction boundaries, idempotency keys, backups, and rollback for reversible operations.
- Set maximum steps, retries, timeouts, token budgets, and monetary budgets.
- Treat webpages, documents, emails, tool descriptions, and tool outputs as untrusted data unless explicitly trusted.
- Log prompts, selected tools, arguments, observations, decisions, approvals, and final outcomes.
- Escalate ambiguity, high impact, missing evidence, or repeated failure to a human.

> **Operational rule**
>
> The model may propose. Deterministic code and authorized humans decide what is permitted, what is executed, and what counts as complete.

8 Putting the sources together

8.1 From language reasoning to interoperable agents

Source contribution	Core idea	Role in a modern agent

Source contribution	Core idea	Role in a modern agent
Wei et al. — CoT	Intermediate language steps can elicit multi-step reasoning.	Plan decomposition and structured intermediate work.
Yao et al. — ReAct	Interleave reasoning, actions, and observations.	Closed-loop grounding, progress tracking, and recovery.
Anthropic — effective agents	Use simple composable patterns and increase autonomy only when justified.	Architecture selection, workflow patterns, ACI design, stopping conditions.
Huyen — tools and planning	Agents depend on tool inventory, planning, validation, reflection, and evaluation.	Engineering decomposition, failure taxonomy, metrics, and human checkpoints.
MCP — connectivity	Standardize how hosts connect to external resources, prompts, and tools.	Reusable integration layer across applications and servers.

8.2 Recommended build sequence

1. Define the task, environment, allowed actions, constraints, and measurable success criteria.
2. Build a single-call baseline with a clear prompt and representative evaluation set.
3. Add retrieval or one specialized tool when the failure is missing knowledge or a narrow capability gap.
4. Use a deterministic workflow when the path can be predefined and validated.
5. Introduce model-directed planning only when environmental feedback makes the required steps unpredictable.
6. Design tools as precise interfaces: clear names, schemas, examples, errors, and permission boundaries.
7. Add plan validation, observation checks, loop detection, budgets, and explicit stopping conditions.
8. Require human approval for consequential writes and preserve rollback or dry-run paths.
9. Instrument the entire trajectory and classify failures by stage.
10. Increase complexity only when evaluation shows a measurable gain in task success, safety, or human time saved.

8.3 Architecture selection guide

Question	If yes	If no
Can one prompt, retrieval step, or tool call solve the task reliably?	Use a single augmented LLM call.	Continue.
Can the subtasks and order be specified in advance?	Use a workflow such as chaining, routing, or parallelization.	Continue.
Do observations change which steps are needed?	Consider a bounded agent loop.	Prefer a workflow.
Can progress and completion be verified automatically?	Allow more autonomy with checkpoints.	Keep a human in the loop.

Question	If yes	If no
Are actions reversible and permissions narrowly scoped?	Permit controlled execution.	Use read-only mode, dry-run, or human execution.
Does measured value exceed added cost and risk?	Deploy incrementally and monitor.	Return to the simpler design.

8.4 Ten key takeaways

1. An agent is a closed-loop system, not merely a long prompt or a model with a persona.
2. Workflows follow predefined code paths; agents dynamically choose steps and tools.
3. Chain-of-thought can elicit decomposition, but visible reasoning is not guaranteed to be faithful or factual.
4. ReAct improves grounding by allowing observations to revise reasoning and action plans.
5. Reasoning-only and acting-only systems have complementary weaknesses; hybrid control can outperform either alone.
6. Tool design and documentation are part of prompt engineering and often determine agent reliability.
7. Planning should be validated before execution, and reflection should verify progress after execution.
8. MCP standardizes connectivity; it does not supply planning, safety, authorization, or evaluation by itself.
9. Long action horizons compound errors, so bounded loops, checkpoints, and deterministic controls are essential.
10. The right agent is the simplest system that meets a measured objective within acceptable cost and risk.

9 Self-check questions

Use these questions to test conceptual understanding and system-design judgment.

1. What architectural property distinguishes an agent from a fixed workflow?
2. Why can a multi-call workflow still be non-agentic?
3. How does chain-of-thought prompting differ from fine-tuning?
4. Why did the CoT ablations suggest that extra output tokens alone were not sufficient?
5. Why is a chain-of-thought trace not proof of faithful internal reasoning?
6. What does “reason to act” mean in ReAct?
7. What does “act to reason” mean in ReAct?
8. Why did ReAct reduce hallucination yet increase some reasoning and search-related failures?
9. When should an orchestrator-workers pattern be preferred over predefined parallelization?
10. What is the difference between a read tool and a write tool?
11. Why must application code validate a model-generated function call before execution?
12. How can plan validation reduce both cost and safety risk?
13. What is the difference between high-level planning and executable planning?
14. Why does reliability decline rapidly as an agent performs more dependent steps?
15. What are the roles of an MCP host, client, and server?
16. How do MCP resources, prompts, and tools differ?

17. Why is MCP not equivalent to an autonomous agent?
18. Name three indicators that an agent has falsely declared completion.
19. Design three metrics for tool-use accuracy and three for end-to-end task success.
20. For a high-impact write action, where should human approval occur and what rollback mechanism should exist?

9.1 Short design exercise

Design an agent that reviews a software repository and proposes a patch. Specify:

- the environment and goal;
- the minimum tool inventory;
- which operations are read-only and which are write actions;
- the planning and validation stages;
- the observation and reflection loop;
- the completion test;
- the human approval checkpoint;
- the failure metrics and cost limits;
- whether MCP would be useful and what servers would be needed.

10 Glossary

Term	Definition
Action	An operation performed by an agent in an environment.
Agent	A system that perceives an environment, chooses actions, observes results, and works toward a goal.
Agent-computer interface (ACI)	The descriptions, schemas, formats, and feedback through which a model interacts with software tools.
Agentic system	A broad category that includes both predefined LLM workflows and model-directed agents.
Chain-of-thought prompting	Few-shot prompting that demonstrates intermediate natural-language reasoning steps before the final answer.
Control flow	The order and conditions under which steps execute: sequential, parallel, conditional, or iterative.
Elicitation	An MCP client capability that lets a server request additional user information through the host.
Environment	The external system or world in which an agent receives observations and takes actions.
Evaluator-optimizer	A workflow in which one model produces an output and another critiques it in an iterative loop.

Term	Definition
Function calling	A model API pattern in which the model proposes a structured tool invocation for application code to validate and execute.
Grounding	Basing claims or actions on external evidence or observations rather than unsupported model memory.
Host	The MCP-enabled AI application that manages users, models, policies, and client connections.
Human in the loop	A design in which humans approve, correct, execute, or judge selected stages.
MCP client	The connector inside a host that communicates with one MCP server.
MCP server	A service that exposes resources, prompts, tools, or other supported capabilities through MCP.
Memory / state	Information retained across steps to track context, progress, and unresolved subgoals.
Observation	Feedback from the environment after an action or tool call.
Orchestrator-workers	A pattern in which a central model dynamically creates subtasks, delegates them, and synthesizes results.
Parallelization	Running independent subtasks or multiple attempts at the same time, then aggregating results.
Plan	A proposed sequence or structure of actions intended to accomplish a goal.
Policy	The mechanism that selects an action from the current context or state.
Prompt chaining	A fixed workflow where one model call processes the output of the previous call.
ReAct	A prompting paradigm that interleaves reasoning traces, task-specific actions, and observations.
Reflection	Evaluation of a plan, action outcome, or overall trajectory to identify errors and decide whether to continue or replan.
Resource	Contextual data exposed by an MCP server for the user or model to read.
Routing	Classifying an input and directing it to a specialized process, prompt, model, or toolset.
Sandbox	An isolated environment that limits the consequences of code or tool execution.
Stopping condition	A rule that terminates an agent loop because the goal, limit, handoff, or safety condition has been reached.
Tool	A callable capability that extends what the model can perceive or do.
Tool inventory	The set of tools available to the agent for a specific task or request.

Term	Definition
Workflow	A system whose model and tool calls follow predefined code paths.
Write action	A tool operation that changes an external system or produces a real-world consequence.

11 Assigned sources and reading notes

The chapter synthesizes the assigned materials. Dates and versions matter because agent frameworks, model capabilities, and protocol specifications change rapidly.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS 2022; arXiv version 6 (2023).
Use for the definition of CoT prompting, scale-dependent results, arithmetic / commonsense / symbolic experiments, ablations, robustness, and limitations.
https://arxiv.org/abs/2201.11903

Yao, S., Zhao, J., Yu, D., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023.
Use for the thought-action-observation loop, reason-to-act / act-to-reason framing, benchmark comparisons, grounding trade-offs, failure analysis, and interactive decision making.
https://arxiv.org/abs/2210.03629

Anthropic. (2024, December 19). "Building Effective Agents."
Use for the workflow-versus-agent distinction, simplicity principle, prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, autonomous agents, and ACI guidance.
https://www.anthropic.com/engineering/building-effective-agents

Huyen, C. (2025, January 7). "Agents."
Use for environment / action definitions, tool categories, function calling, planning and validation, reflection, tool selection, compounding error, agent failure modes, and evaluation metrics.
https://huyenchip.com/2025/01/07/agents.html

Model Context Protocol. (2026). "What Is the Model Context Protocol (MCP)?" and MCP Specification, version 2026-07-28.
Use for the host-client-server architecture, resources / prompts / tools / elicitation, stateless protocol details, extensions, interoperability, and security principles.
https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro